The collections module in Python provides specialized container datatypes that are alternatives to Python's general-purpose built-in containers like list, dict, and tuple. Below is a detailed explanation of the most commonly used classes in the collections module, along with examples and code snippets.

---

1. deque (Double-Ended Queue)

A deque is a list-like container with fast appends and pops from both ends.

Example:

```
from collections import deque

# Create a deque
d = deque([1, 2, 3])

# Append to the right
d.append(4)  # deque([1, 2, 3, 4])

# Append to the left
d.appendleft(0)  # deque([0, 1, 2, 3, 4])

# Pop from the right
d.pop()  # Returns 4, deque([0, 1, 2, 3])

# Pop from the left
d.popleft()  # Returns 0, deque([1, 2, 3])


# Rotate the deque
d.rotate(1)  # deque([3, 1, 2])
```

---

2. Counter

A Counter is a dictionary subclass for counting hashable objects.

Example:

```
from collections import Counter

# Create a Counter
c = Counter("hello")


# Count elements
print(c)  # Counter({'h': 1, 'e': 1, 'l': 2, 'o': 1})


# Most common elements
print(c.most_common(2))  # [('l', 2), ('h', 1)]


# Update Counter
c.update("world")
print(c)  # Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, 'w': 1, 'r': 1, 'd': 1})
```

---

3. defaultdict

A defaultdict is a dictionary subclass that provides a default value for missing keys.

Example:

```
from collections import defaultdict

# Create a defaultdict with default value as 0
d = defaultdict(int)
# Add elements
d['a'] = 1
d['b'] = 2
```

```
# Access a missing key
print(d['c'])  # Returns 0 (default value)
```

---

4. namedtuple

A namedtuple creates tuple-like objects with named fields.

Example:

```
from collections import namedtuple

# Create a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Access fields
print(p.x)  # 10
print(p.y)  # 20

# Unpack like a regular tuple
x, y = p
print(x, y)  # 10 20
```

---

5. ChainMap

A ChainMap groups multiple dictionaries into a single mapping.

Example:

```
from collections import ChainMap

# Create dictionaries
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}

# Create a ChainMap
chain = ChainMap(dict1, dict2)

# Access elements
print(chain['a'])  # 1 (from dict1)
print(chain['b'])  # 2 (from dict1, first match)
print(chain['c'])  # 4 (from dict2)
```

---

6. OrderedDict

An OrderedDict is a dictionary subclass that maintains the order of keys.

Example:

```
from collections import OrderedDict
# Create an OrderedDict
od = OrderedDict()
# Add elements
od['a'] = 1
od['b'] = 2
```

```
od['c'] = 3


# Iterate in insertion order
for key, value in od.items():
    print(key, value)  # a 1, b 2, c 3
```

---

7. UserDict, UserList, UserString

These are wrapper classes for creating custom dictionary-like, list-like, and string-like objects.

Example:

```
from collections import UserDict


# Create a custom dictionary
class MyDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key, value * 2)


# Use the custom dictionary
d = MyDict()
d['a'] = 1
print(d)  # {'a': 2}
```

---

8. Counter Example: Word Frequency

```
from collections import Counter
```

```
# Count word frequency in a sentence
sentence = "the quick brown fox jumps over the lazy dog"
words = sentence.split()
word_count = Counter(words)

print(word_count)
# Output: Counter({'the': 2, 'quick': 1, 'brown': 1, 'fox': 1, 'jumps': 1, 'over': 1, 'lazy': 1, 'dog': 1})
```

---

9. defaultdict Example: Grouping

```
from collections import defaultdict

# Group names by their first letter
names = ["Alice", "Bob", "Charlie", "David", "Eve"]
grouped_names = defaultdict(list)

for name in names:
    grouped_names[name[0]].append(name)

print(grouped_names)
# Output: defaultdict(<class 'list'>, {'A': ['Alice'], 'B': ['Bob'], 'C': ['Charlie'], 'D': ['David'], 'E': ['Eve']})
```

---

10. deque Example: Sliding Window Maximum

```
from collections import deque

def sliding_window_maximum(nums, k):
    dq = deque()
    result = []

    for i, num in enumerate(nums):
        # Remove indices of elements not in the current window
        while dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove indices of smaller elements
        while dq and nums[dq[-1]] < num:
            dq.pop()

        dq.append(i)

        # Append the maximum for the current window
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result
```

```
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3
print(sliding_window_maximum(nums, k))
# Output: [3, 3, 5, 5, 6, 7]
```

---

1. Counter Example: Finding the Most Common Elements

```
from collections import Counter
# Count the frequency of elements in a list
data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
counter = Counter(data)

# Find the 2 most common elements
print(counter.most_common(2))  # Output: [(4, 4), (3, 3)]
```

---

2. defaultdict Example: Counting Characters in a String

```
from collections import defaultdict

# Count characters in a string
s = "hello world"
char_count = defaultdict(int)

for char in s:
    char_count[char] += 1
```

```
print(char_count)

# Output: defaultdict(<class 'int'>, {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1})
```

---

3. namedtuple Example: Representing a Point in 3D Space

```
from collections import namedtuple

# Define a namedtuple for 3D points
Point3D = namedtuple('Point3D', ['x', 'y', 'z'])


# Create a 3D point
p = Point3D(1, 2, 3)


# Access fields
print(p.x, p.y, p.z)  # Output: 1 2 3
```

---

4. ChainMap Example: Merging Dictionaries

```
from collections import ChainMap
# Create dictionaries
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
dict3 = {'d': 5}


# Merge dictionaries using ChainMap
merged = ChainMap(dict1, dict2, dict3)
```

```
# Access elements
print(merged['a'])  # Output: 1 (from dict1)
print(merged['b'])  # Output: 2 (from dict1, first match)
print(merged['d'])  # Output: 5 (from dict3)
```

---

5. OrderedDict Example: Maintaining Insertion Order

```
from collections import OrderedDict
# Create an OrderedDict
od = OrderedDict()
# Add elements
od['a'] = 1
od['b'] = 2
od['c'] = 3

# Move 'b' to the end
od.move_to_end('b')

# Iterate in order
for key, value in od.items():
    print(key, value)
# Output: a 1, c 3, b 2
```

---

6. deque Example: Implementing a Queue

```
from collections import deque


# Create a queue
queue = deque()


# Enqueue elements
queue.append(1)
queue.append(2)
queue.append(3)


# Dequeue elements
print(queue.popleft())  # Output: 1
print(queue.popleft())  # Output: 2
```

---

7. Counter Example: Subtracting Counts

```
from collections import Counter
# Create two Counters
c1 = Counter(a=3, b=2, c=1)
c2 = Counter(a=1, b=2, c=3)


# Subtract counts
c1.subtract(c2)


print(c1)  # Output: Counter({'a': 2, 'b': 0, 'c': -2})
```

---

8. defaultdict Example: Grouping by Length

```
from collections import defaultdict

# Group words by their length
words = ["apple", "bat", "bar", "atom", "book"]
grouped_words = defaultdict(list)

for word in words:
    grouped_words[len(word)].append(word)

print(grouped_words)
# Output: defaultdict(<class 'list'>, {5: ['apple'], 3: ['bat', 'bar'], 4: ['atom', 'book']})
```

---

9. namedtuple Example: Representing a RGB Color

```
from collections import namedtuple

# Define a namedtuple for RGB colors
RGB = namedtuple('RGB', ['red', 'green', 'blue'])

# Create a color
color = RGB(255, 0, 0)
```

```
# Access fields
print(color.red, color.green, color.blue)  # Output: 255 0 0
```

---

10. ChainMap Example: Updating Values

```
from collections import ChainMap

# Create dictionaries
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}

# Create a ChainMap
chain = ChainMap(dict1, dict2)

# Update a value
chain['b'] = 5

print(chain)
# Output: ChainMap({'a': 1, 'b': 5}, {'b': 3, 'c': 4})
```

---

11. OrderedDict Example: Reversing Order

```
from collections import OrderedDict
# Create an OrderedDict
od = OrderedDict([('a', 1), ('b', 2), ('c', 3)])
```

```
# Reverse the order
reversed_od = OrderedDict(reversed(od.items()))


print(reversed_od)
# Output: OrderedDict([('c', 3), ('b', 2), ('a', 1)])
```

---

12. deque Example: Implementing a Stack

```
from collections import deque
# Create a stack
stack = deque()


# Push elements
stack.append(1)
stack.append(2)
stack.append(3)


# Pop elements
print(stack.pop())  # Output: 3
print(stack.pop())  # Output: 2
```

---

13. Counter Example: Finding the Least Common Elements

```
from collections import Counter
# Count the frequency of elements in a list
data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
```

```
counter = Counter(data)

# Find the 2 least common elements
print(counter.most_common()[:-3:-1])  # Output: [(1, 1), (2, 2)]
```

---

14. defaultdict Example: Counting Words in a File

```
from collections import defaultdict

# Count words in a file
word_count = defaultdict(int)

with open("example.txt", "r") as file:
    for line in file:
        for word in line.split():
            word_count[word] += 1

print(word_count)
```

---

15. namedtuple Example: Representing a Date

```
from collections import namedtuple
# Define a namedtuple for dates
Date = namedtuple('Date', ['year', 'month', 'day'])

# Create a date
```

```
today = Date(2023, 10, 5)


# Access fields
print(today.year, today.month, today.day)  # Output: 2023 10 5
```

---

1. deque (Double-Ended Queue)

- **Purpose**: A list-like container with fast appends and pops from both ends.
- **Use Case**: Ideal for implementing queues, stacks, and sliding window algorithms.
- **Example**:

```
from collections import deque
d = deque([1, 2, 3])
d.append(4)  # Add to the right
d.appendleft(0)  # Add to the left
print(d)  # Output: deque([0, 1, 2, 3, 4])
```

---

2. Counter

- **Purpose**: A dictionary subclass for counting hashable objects.
- **Use Case**: Counting frequencies of elements in a list, finding most common elements, etc.
- **Example**:

```
from collections import Counter
c = Counter("hello")
print(c)  # Output: Counter({'h': 1, 'e': 1, 'l': 2, 'o': 1})
```

---

3. defaultdict

- **Purpose**: A dictionary subclass that provides a default value for missing keys.
- **Use Case**: Simplifies code when working with dictionaries where keys might not exist.
- **Example**:

```
from collections import defaultdict

d = defaultdict(int)

d['a'] += 1

print(d)  # Output: defaultdict(<class 'int'>, {'a': 1})
```

---

4. namedtuple

- **Purpose**: A factory function for creating tuple subclasses with named fields.
- **Use Case**: Makes code more readable by giving meaning to each position in a tuple.
- **Example**:

```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])

p = Point(10, 20)

print(p.x, p.y)  # Output: 10 20
```

---

5. OrderedDict

- **Purpose**: A dictionary subclass that maintains the order of keys.
- **Use Case**: Useful when the insertion order of keys matters.
- **Example**:

```
from collections import OrderedDict
```

```
od = OrderedDict()
od['a'] = 1
od['b'] = 2
print(od)  # Output: OrderedDict([('a', 1), ('b', 2)])
```

---

6. ChainMap

- **Purpose**: A class for grouping multiple dictionaries into a single mapping.
- **Use Case**: Useful for combining multiple dictionaries without merging them.
- **Example**:

```
from collections import ChainMap
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
chain = ChainMap(dict1, dict2)
print(chain['a'])  # Output: 1
```

---

7. UserDict, UserList, UserString

- **Purpose**: Wrapper classes for creating custom dictionary-like, list-like, and string-like objects.
- **Use Case**: Useful for extending or customizing the behavior of built-in containers.
- **Example**:

```
from collections import UserDict
class MyDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key, value * 2)
```

```
d = MyDict()
d['a'] = 1
print(d)  # Output: {'a': 2}
```

---

8. abc (Abstract Base Classes)

- **Purpose**: Provides abstract base classes for custom container types.
- **Use Case**: Useful for defining custom containers that adhere to specific interfaces.
- **Example**:

```
from collections.abc import MutableSequence
class MyList(MutableSequence):
    def __init__(self, data):
        self.data = list(data)
    def __getitem__(self, index):
        return self.data[index]
    def __len__(self):
        return len(self.data)
    def __setitem__(self, index, value):
        self.data[index] = value
    def __delitem__(self, index):
        del self.data[index]
    def insert(self, index, value):
        self.data.insert(index, value)
my_list = MyList([1, 2, 3])
```

```
print(my_list[0])  # Output: 1
```

---

9. Counter Example: Finding the Least Common Elements

- **Purpose**: Extends the functionality of Counter to find the least common elements.
- **Use Case**: Useful for analyzing data distributions.
- **Example**:

```
from collections import Counter

c = Counter("hello")

print(c.most_common()[:-3:-1])  # Output: [('e', 1), ('h', 1)]
```

---

10. deque Example: Sliding Window Maximum

- **Purpose**: Demonstrates the use of deque for solving algorithmic problems.
- **Use Case**: Efficiently finding the maximum in a sliding window.
- **Example**:

```
from collections import deque

def sliding_window_maximum(nums, k):
    dq = deque()
    result = []
    for i, num in enumerate(nums):
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        while dq and nums[dq[-1]] < num:
            dq.pop()
```

```
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
nums = [1, 3, -1, -3, 5, 3, 6, 7]
print(sliding_window_maximum(nums, 3))  # Output: [3, 3, 5, 5, 6, 7]
```

Summary of Important Classes

Class	Purpose
deque	Fast appends and pops from both ends.
Counter	Counts hashable objects.
defaultdict	Provides default values for missing keys.
namedtuple	Creates tuple-like objects with named fields.
OrderedDict	Maintains the order of keys in a dictionary.
ChainMap	Groups multiple dictionaries into a single mapping.
UserDict	Wrapper for creating custom dictionary-like objects.
UserList	Wrapper for creating custom list-like objects.
UserString	Wrapper for creating custom string-like objects.
